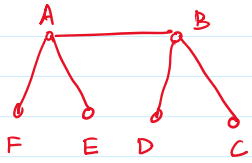
We build the solution incrementally. We enlarge the solution using greedy approach.

Independent Set $G=(V,E)$ be a graph. A subset $V' \subseteq V$ is called indep set

if $\forall\ x,y \in V' \quad (x,y) \notin E$

↑ edge



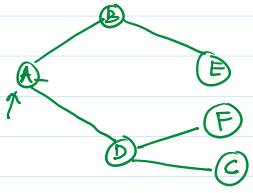
Indepset $\{F,E,D,C\}$ $\{A,C,D\}$ $\{F,E,B\}$ ..

Problem Given a graph $G=(V,E)$ find indep set of largest size.

For graph the problem is NP-Hard.

We restrict our input to tree only.

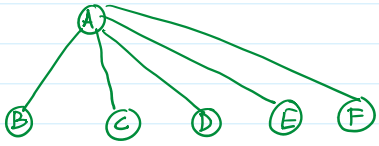
Problem: Input: Tree T (connected and acyclic)
Output: Max Indep set in T



Alternate Approach

① Pick a vertex. Put it in indep set

② Remove all its neighbours (because they can't be in indep set)



if we pick A, then indep set $=\{A\}$

OPT soln $\{B,C,D,E,F\}$

⇒ Should pick the vertex of smallest degree – leaves.

ALG

Start with all the leaves (put them in indep set)

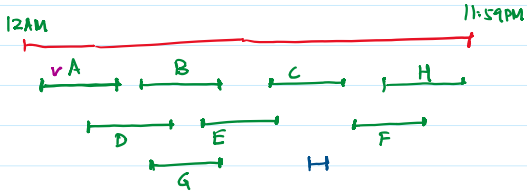
Remove their neighbours.

Repeat

Problem Input: Set of Intervals

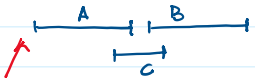
Output: Subset of non-overlapping interval of max size

Key: which Interval should we pick first.



Set of Intervals $\{A, B, C \ldots G\}$
Soln $\{A, B, C, H\}$

ALG 1: Pick the Interval of smallest length

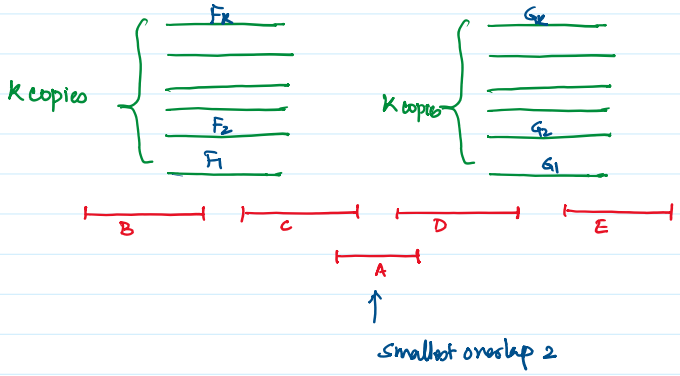


ALG1 fails on this ex. OPT = 2 ALG1 output 1

ALG2: Pick the Interval which overlaps with smallest no. of Intervals.
(This is related to indep set problem.)

- In Indep Set we pick leaves, throw away all its neighbours and repeat
- In this problem, we pick interval (which overlaps with smallest no. of intervals), throw away all the intervals which overlaps, and repeat

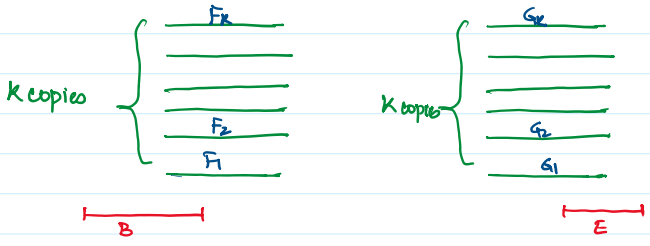
Intervals	# of overlap
A	2
B	K
F_i	(K+1)
E	K
G_i	(K+1)
C, D	(K+1)



Soln set = $\{A\}$

Intervals	# of overlap
B	K
F_i	K
E	K
G_i	K

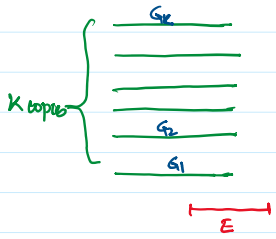
Updated Input



Soln Set $\{A, B\}$

Intervals	# of overlap
E	K
G_1	K

Updated Input

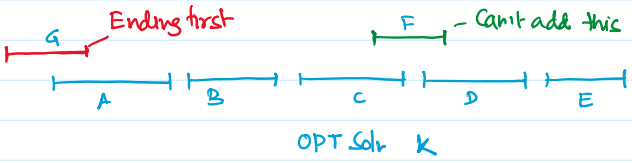


Soln Set $\{A, B, E\}$

- Alg 2 outputs 3 intervals but OPT is 4 $\{B, C, D, E\}$

Does not work !!

How does opt Soln look like !!

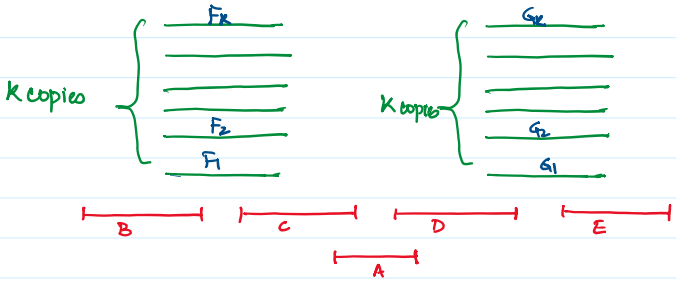


We can always add one interval that ends first. If needed we can remove the first interval (A in the example). The size of Soln set doesn't go down.

(1)

Interval B ends first

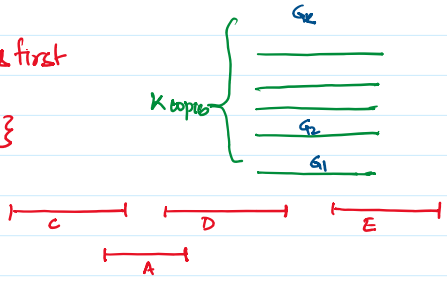
Soln Set = $\{B\}$



(2)

Interval C ends first

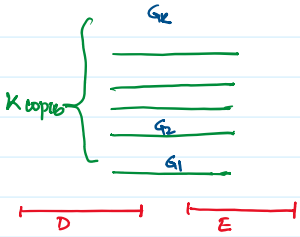
Soln Set = $\{B, C\}$



(3)

Interval D ends first

Soln Set = $\{B, C, D\}$





④

Internal E ends first

Soln Set = {B, C, D, E}



ALG 3
- Add the internal that ends first
- Remove the internals that overlap with this
- Repeat

Proof of correctness: Look at any opt Soln. If it contains the internal that ends first, we are done. If not, add the internal that ends first, and remove the overlapping internal (Add A and remove B, in this ex)



The size of soln will not go down.
Recurse on remaining internal. By induction proof is complete.

Information Transmission

Input: [Symbols x_1 x_2 - - x_n ; frequency f_1 f_2 - - f_n] Input

Output: Binary codes C_1 C_2 - - C_n] Output C_i := Binary string

For eg If we get code $C_1 C_2 C_1$ ⇒ we have sent symbols $x_1 x_2 x_1$

Ex

Symbol	freq	Code
A	100	0
B	2	10
C	2	11

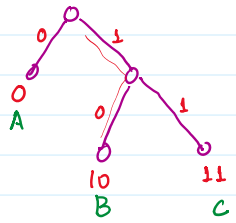
A = 11
B = 111

- We want the codeword to be prefix free eg A = 11 then B ≠ 111.
  C_i should not be prefix of C_j for any $i \neq j$
- A has sent multiple times, we should give small sx code

11 00 10 00 10 11
C AA B AA B C

① How to generate prefix free code

② what is overall also s.t overall message length is minimized.



Each leaf gives code of symbol

A = 0
B 10
C 11

To generate prefix free code for n symbol, Generate a Binary tree with n leaves

Each left edge is label as 0 and right marked as 1. The label of leaf node the concatenation of labels of edges sitting on path from root to that leaf.

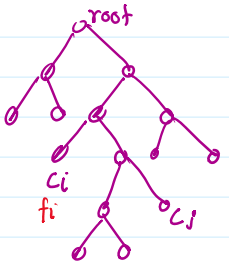
Claim This coding is prefix free.

Proof: Take any node in the leaf c_i (leaf node). c_i is not prefix of any c_j $i \neq j$ some other leaf node

c_i has some binary code associated to it.

c_i does not sit on the path from root to c_j.

$\Rightarrow$ Code word of c_i is not prefix of c_j $\forall\ i,j \in [n]$.



Final Objective: Given these symbols and frequencies, the aim is to construct prefix free binary code st overall message length is minimized.

Input $f_1 - - f_n$ freq
$x_1 - - x_n$ Symbol

Output Prefix free code $\leftrightarrow$ Binary tree st minimize $\sum_{i=1}^{n} h_i f_i$ — (1)

h_i depth of c_i in Binary tree

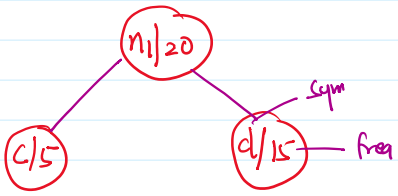
There are many binary tree possible that has n leaves.

[Suppose if correct tree is given, then sort the symbols based on their freq and element with higher freq will be closest to the root]

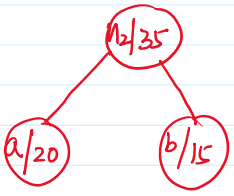
Set of alphabets Σ.

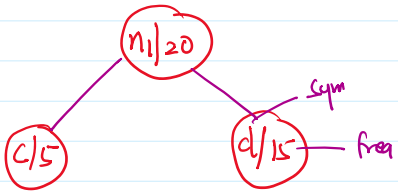
Symbol	x	y - -
freq	f_x	f_y

Sym	a	b	c	d	e
freq	20	15	5	15	45

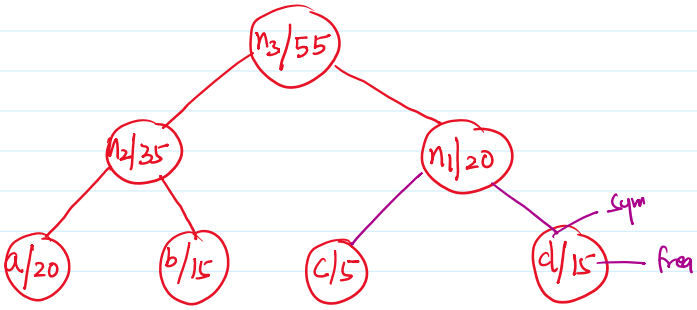


Sym	a	b	n_1	e
freq	20	15	20	45





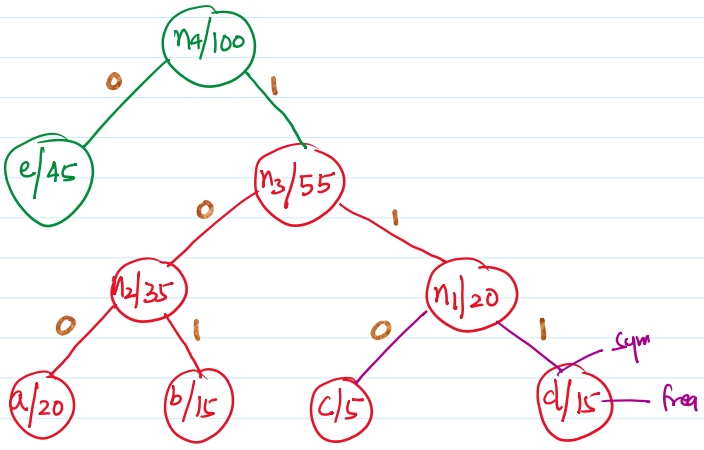
Sym	n_2	x	n_1	e
freq	35	x	20	45



Sym	n_3	e
freq	55	45

Sym	n_3	e
freq	55	45

Symbol	Codeword	freq
e	0	45
a	100	20
b	101	15
c	110	5
d	111	15



Step 1: Pick two symbols (say x & y) having smallest freq. Create subtree with these two symbols as leaves. Generate new node n_i
make n_i as parent of x & y.
Assign freq n_i as sum of fx & fy



Step2 Remove x & y and add n_i in Σ

$$\Sigma' \leftarrow (\Sigma - \{x, y\}) \cup \{n_i\}$$

Repeat Step1

Huffman Coding — David A Huffman

[Robert Fano
Shannon

Theorem ALG is OPT.

Homework